PYTHON PROGRAM
C0_5

NAME: SHABANA THASLEEM A.W

COURSE: PYTHON PROGRAMMING

COURSE CODE: CSA0802

ASSIGNMENT: 5

NATURE OF THE ASSIGNMENT: Individual

TOTAL MARKS: 50

SUBMISSION DATE: 12/11/26

PYTHON PROGRAM
C0_5

PART A – DATA AND PROGRAM DESIGN (25 MARKS)

TASK A1 – DESIGN THE DATA ACQUISITION (7 MARKS)

A1.1 – Inspect the Source (2 Marks)

Question: Describe the structure of a station CSV file and list every data-quality problem expected, with an example of each.

Answer

The station CSV file contains air-quality readings collected from different monitoring stations.

Expected CSV format:

```
station,date,pm25
Adyar,2026-10-01,48
Adyar,2026-10-02,NA
Adyar,2026-10-03,
Guindy,2026-10-01,95
Guindy,2026-10-02,-5
Guindy,2026-10-03,121
```

Description of columns:

Column	Data type	Description
station	String	Name of monitoring station
date	Date	Date of measurement
pm25	Float	PM2.5 concentration

Data-quality problems

No.	Problem	Example
1	Missing values	Empty PM2.5 value
2	NA values	`NA` instead of a number
3	Negative values	`-5`
4	Duplicate records	Same station and date repeated
5	Wrong date format	`01/10/2026` instead of `2026-10-01`
6	Invalid station name	`Adyar` with extra spaces

7	Non-numeric values	`high` instead of `48`
8	Extreme values	PM2.5 greater than 999
9	Wrong encoding	Unsupported characters in station names
10	Wrong delimiter	Semicolon instead of comma

Conclusion: Data validation is necessary before performing statistical analysis because incorrect data can produce misleading results.

A1.2 – Design the Storage (3 Marks)

Question: Design SQLite tables for stations and readings, including columns, data types and primary keys.

Answer

SQLite is suitable for storing air-quality readings because it supports structured data, SQL queries and indexing.

Database design:

Table 1 – Stations

Column	Data type	Constraint
station_id	INTEGER	PRIMARY KEY
station_name	TEXT	UNIQUE, NOT NULL

Table 2 – Readings

Column	Data type	Constraint
reading_id	INTEGER	PRIMARY KEY
station_id	INTEGER	FOREIGN KEY
date	TEXT	NOT NULL
pm25	REAL	CHECK (pm25 >= 0)

SQL table creation

```
CREATE TABLE stations (
    station_id INTEGER PRIMARY KEY,
    station_name TEXT UNIQUE NOT NULL
```

```
);

CREATE TABLE readings (
    reading_id INTEGER PRIMARY KEY,
    station_id INTEGER NOT NULL,
    date TEXT NOT NULL,
    pm25 REAL NOT NULL CHECK(pm25 >= 0),

    FOREIGN KEY (station_id)
    REFERENCES stations(station_id),

    UNIQUE(station_id, date)
);
```

CSV columns mapping

CSV column	SQLite column
station	station_name in stations
date	date in readings
pm25	pm25 in readings

The `station_id` connects the two tables using a foreign key.

Advantages:

1. Avoids repeated station names.
2. Prevents duplicate station-date readings.
3. Supports efficient SQL queries.
4. Maintains data consistency.

A1.3 – CSV or SQL? (2 Marks)

Question: Compare answering three typical questions using CSV files and SQL queries.

Three questions

1. What is the average PM2.5 of Adyar station in October?
2. Which station has the highest PM2.5?

PYTHON PROGRAM
C0_5

3. What is the average PM2.5 for each station?

Method 1 – Using CSV

```python
import csv

values = []

with open("readings.csv", "r") as file:
    reader = csv.DictReader(file)

    for row in reader:
        if row["station"] == "Adyar":
            if row["pm25"] not in ("", "NA"):
                values.append(float(row["pm25"]))

if values:
    print("Average:", sum(values) / len(values))
```

```sql
SELECT AVG(r.pm25)
FROM readings r
JOIN stations s
ON r.station_id = s.station_id
WHERE s.station_name = 'Adyar'
AND r.date BETWEEN '2026-10-01' AND '2026-10-31';
```

Find the highest station reading:

```sql
SELECT s.station_name, MAX(r.pm25)
FROM readings r
JOIN stations s
ON r.station_id = s.station_id
GROUP BY s.station_name;
```

Average PM2.5 for each station:

```sql
SELECT s.station_name, AVG(r.pm25)
FROM readings r
JOIN stations s
```

```
ON r.station_id = s.station_id
GROUP BY s.station_name;
```

Comparison

CSV	SQL
Simple file storage	Structured database storage
Requires manual filtering	Uses SQL queries
Repeated scanning	Can use indexes
Suitable for small datasets	Suitable for larger datasets
Limited concurrent access	Better support for structured multi-user access

Alternative Concept and Complexity Analysis

If CSV files were loaded into a pandas DataFrame for every question:

- Loading and scanning the file takes O(n).
- Each question may require another full scan.
- Memory usage is O(n).

With SQLite:

- An unindexed query may take O(n).
- An indexed lookup can take approximately O(log n + k), where k is the number of matching records.
- Database storage is maintained on disk, reducing the need to load the complete dataset into memory.

Conclusion: CSV is suitable for exchanging small datasets, while SQLite is better for repeated queries and long-term storage.

TASK A2 – DESIGN THE CLEANING AND NORMALISATION (5 MARKS)

A2.1 – Cleaning Rules (2 Marks)

Question: Write a rule for each problem: missing values, impossible values, duplicate rows and Fahrenheit temperatures.

PYTHON PROGRAM
C0_5

Answer

Problem	Cleaning rule	Reason
Missing PM2.5	Forward-fill previous valid reading; use median if no previous reading exists	Avoids unnecessary data loss
Impossible values	Reject negative values and values above 999	Removes invalid readings according to the project's validation rule
Duplicate rows	Keep only one record for the same station and date	Prevents double counting
Fahrenheit temperature	Convert into Celsius	Maintains a standard unit

Fahrenheit conversion formula:

$$ C = \frac{5}{9}(F-32) $$

Example:

$$ C = \frac{5}{9}(98.6-32)=37 $$

Python code:

```
def fahrenheit_to_celsius(f):    return (f - 32) * 5 / 9
```

A2.2 – Normalisation (2 Marks)

1. Min-Max Normalisation

Min-max normalization scales the values between 0 and 1.

Formula:

$$ X'=\frac{X-X_{\min}}{X_{\max}-X_{\min}} $$

Example values:

48, 95, 121

Minimum = 48

Maximum = 121

Results:

Original value	Normalised value
48	0.000
95	0.644
121	1.000

Python code:

```
def min_max(values):    minimum = min(values)    maximum = max(values)
if maximum == minimum:        return [0.0 for value in values]
return [        round((v - minimum) / (maximum - minimum), 3)
for v in values    ]
```

2. Z-Score Standardisation

Formula:

\[Z=\frac{X-\mu}{\sigma} \]

Where:

- X = Original value
- μ = Mean
- σ = Standard deviation

Python code:

```
import statisticsdef z_score(values):    mean =
statistics.mean(values)    std = statistics.pstdev(values)    if std
== 0:        return [0.0 for value in values]    return [(v - mean) /
std for v in values]
```

When to use:

Min-Max	Z-Score
Scales values between 0 and 1	Centres values around 0
Useful for bounded visualisations	Useful for statistical analysis
Sensitive to extreme values	Also affected by extreme values

A2.3 – Formatting (1 Mark)

Standard formats for cleaned data:

PYTHON PROGRAM
C0_5

Field	Standard format	Example
Date	YYYY-MM-DD	2026-10-01
Station name	Title case, trimmed	Adyar
Register number	Not applicable	—
PM2.5	Non-negative decimal	48.0
Temperature	Celsius	37.0

Conclusion: Standard formatting ensures consistency and makes database queries easier.

Alternative Concept and Complexity Analysis

If missing values were dropped instead of filled:

- The number of available readings would decrease.
- The dataset could become biased if missing readings occur more frequently during certain conditions.

Time complexity:

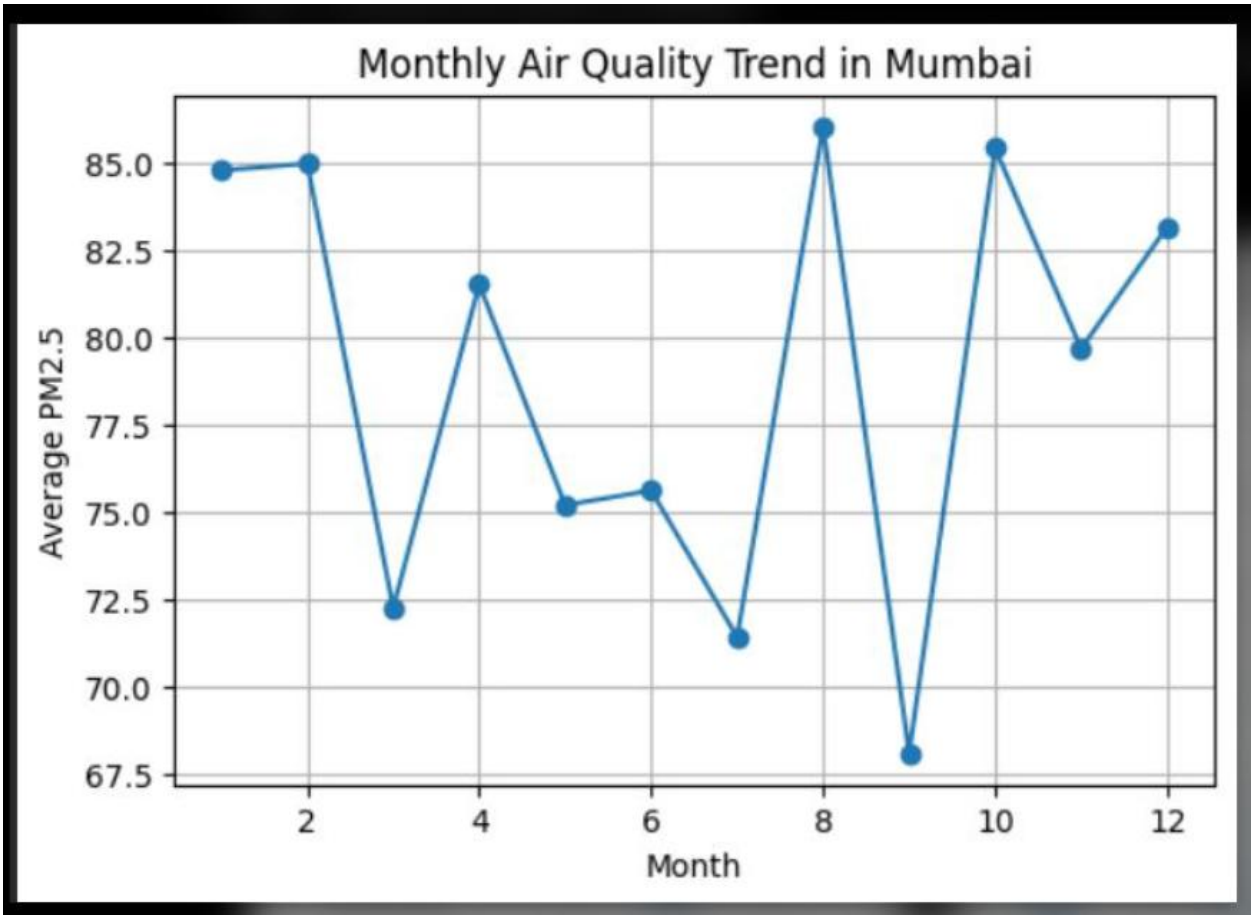
Operation	Complexity
Dropping invalid records	O(n)
Forward filling	O(n)
Median calculation by sorting	O(n log n)
Space for storing cleaned records	O(n)

TASK A3 – DESIGN THE ANALYSIS AND VISUALISATION (5 MARKS)

A3.1 – Choose the Charts (3 Marks)

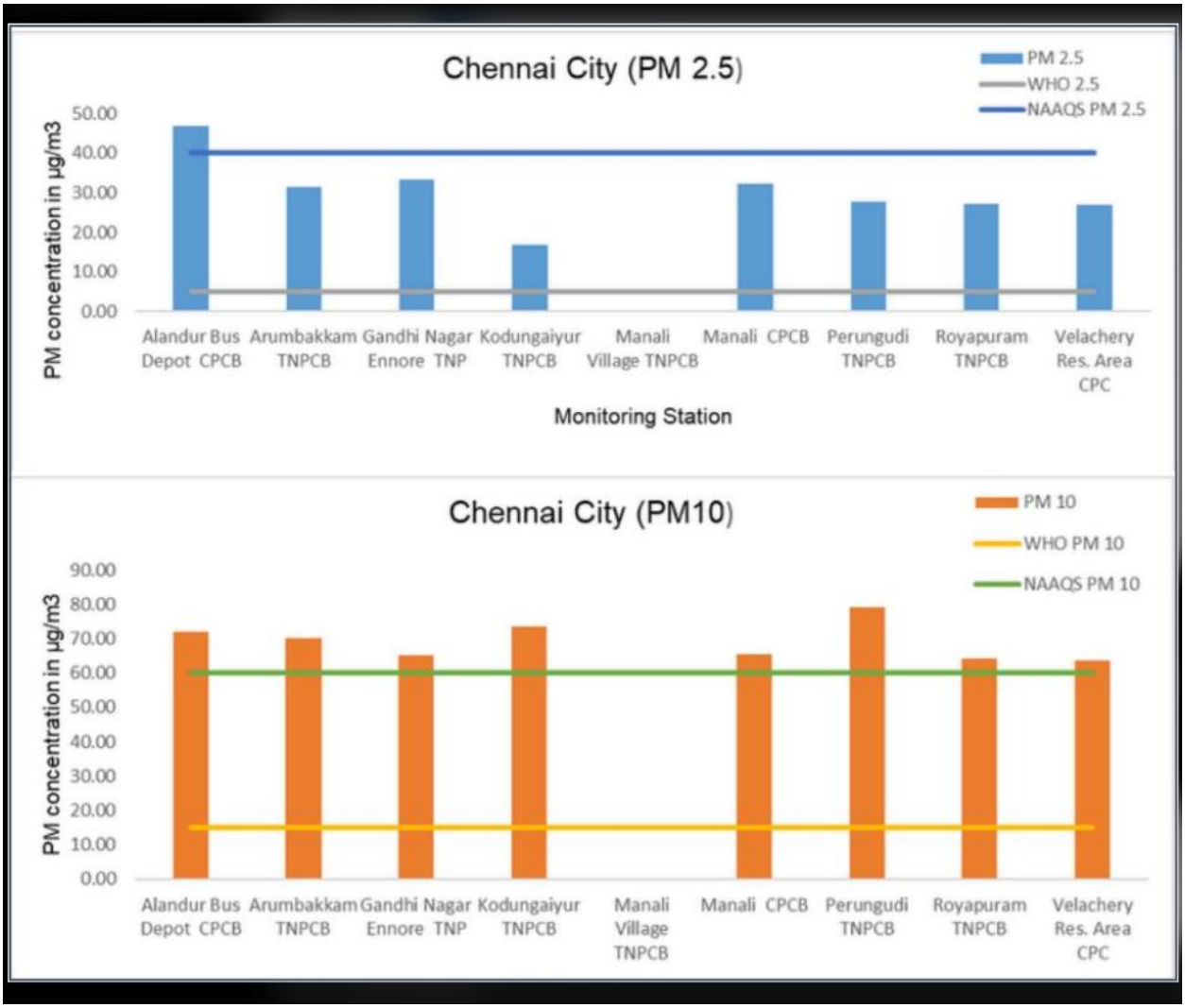
1. Trend of PM2.5 over a month

PYTHON PROGRAM
C0_5



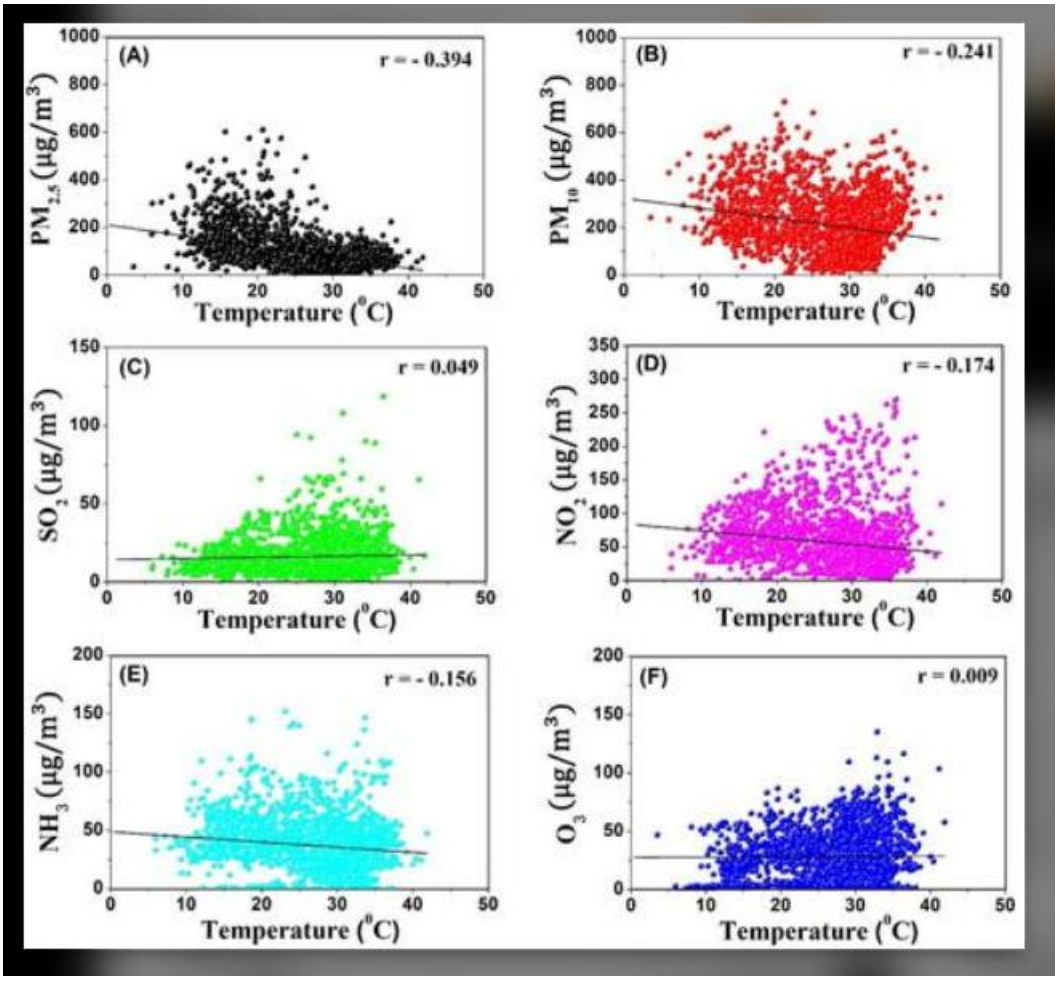
2. Comparison of stations

PYTHON PROGRAM
C0_5



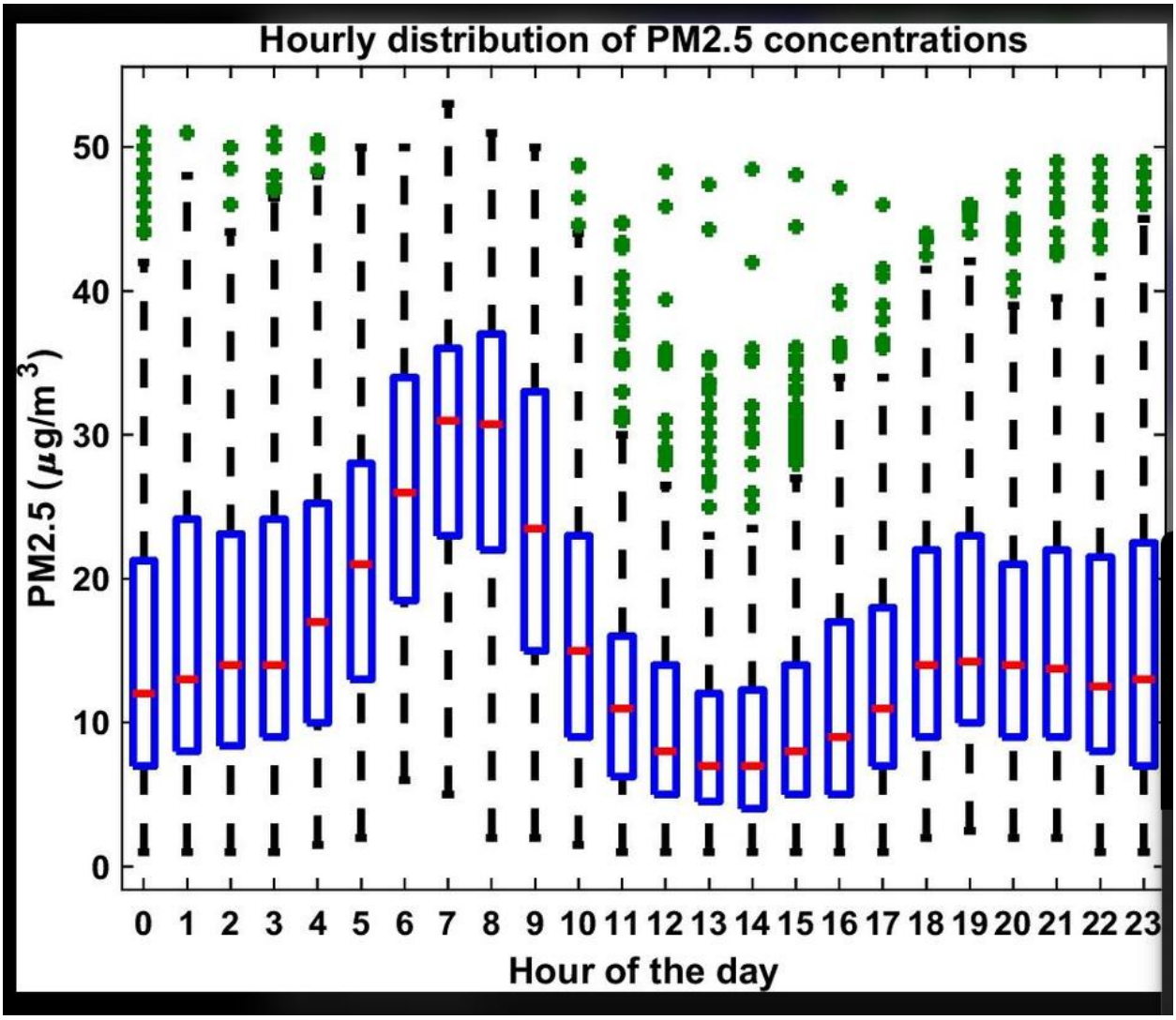
3. Relationship between temperature and PM2.5

PYTHON PROGRAM
C0_5



4. Hours with the worst air

PYTHON PROGRAM
C0_5



A3.2 – Design the Summary Table (2 Marks)

Question: Design a daily summary table per station containing minimum, maximum, mean and number of valid readings.

Answer

Summary table format:

Station	Date	Minimum	Maximum	Mean	Valid readings
Adyar	2026-10-01	48	48	48	1
Guindy	2026-10-01	95	95	95	1
Guindy	2026-10-03	121	121	121	1

SQL query

```
SELECT
    s.station_name,
    r.date,
    MIN(r.pm25) AS minimum,
    MAX(r.pm25) AS maximum,
    AVG(r.pm25) AS mean,
    COUNT(r.pm25) AS valid_readings

FROM readings r

JOIN stations s
ON r.station_id = s.station_id

GROUP BY s.station_name, r.date

ORDER BY r.date, s.station_name;
```

Alternative Concept and Complexity Analysis

If the daily summary were calculated using Python loops instead of SQL `GROUP BY`:

- Both approaches need to process the readings.
- Python requires maintaining aggregation dictionaries.
- SQL performs grouping within the database.

Time complexity:

- Python aggregation: O(n) average.
- SQL grouping: approximately O(n) for hashing, or O(n log n) for sorting-based grouping.
- Memory: O(g), where g is the number of station-date groups.

Conclusion: SQL `GROUP BY` is suitable for producing daily summaries directly from the database.

TASK A4 – PLAN DEBUGGING AND PROFILING (4 MARKS)

A4.1 – Debugging Plan (2 Marks)

Question: Explain how print statements, assert statements, logging and the debugger are used.

1. Print statements

Print statements display intermediate values.

```
print("Original readings:", values)print("Cleaned readings:", clean)
```

They help identify incorrect values during processing.

2. Assert statements

Assertions check whether an expected condition is true.

```
assert all(value >= 0 for value in values)assert len(clean) <=
len(values)
```

If a condition is false, an `AssertionError` occurs.

3. Logging

The logging module records program events and errors.

```
import logginglogging.basicConfig(    filename="debug.log",
level=logging.INFO,
```

```
format="%(levelname)s:%(message)s")logging.info("Cleaning
started")logging.warning("Missing PM2.5 value")logging.error("Invalid
file format")
```

4. Python debugger

The `pdb` debugger helps execute a program step by step.

```
import pdbpdb.set_trace()
```

Useful commands:

Command	Purpose
`n`	Next line
`s`	Step into function
`c`	Continue
`p variable`	Print variable
`q`	Quit debugger

A4.2 – Profiling Plan (2 Marks)

Question: Explain how `time.perf_counter()`, `timeit` and `cProfile` are used.

1. time.perf_counter()

Measures elapsed execution time.

```
import timestart = time.perf_counter()# Execute cleaning functionclean
= load_clean(RAW)end = time.perf_counter()print("Time taken:", end -
start)
```

2. timeit

Measures execution time repeatedly for accurate comparison.

```
import timeitexecution_time = timeit.timeit(    "sum(range(1000))",
number=1000)print(execution_time)
```

3. cProfile

Identifies which functions consume the most execution time.

```
import cProfilecProfile.run("main()")
```

Comparison:

Tool	Purpose
`time.perf_counter()`	Measures elapsed time
`timeit`	Repeated timing measurements
`cProfile`	Identifies slow functions

Alternative Concept and Complexity Analysis

Using print statements for every record can slow down a large program because thousands of additional output operations are performed.

The logging module allows debug messages to be disabled or filtered in production.

Conclusion: Use logging for application diagnostics and profiling tools to identify performance bottlenecks. Disable verbose debugging before production deployment.

TASK A5 – DESIGN AN AI APPLICATION FOR CITIZEN COMPLAINTS (4 MARKS)

A5.1 – Sentiment and Summary (2 Marks)

Question: Design a word-list-based sentiment scorer that handles negation and an extractive summariser that selects sentences containing the most frequent important words.

1. Sentiment analysis

Sentiment analysis identifies whether a citizen complaint expresses a positive, negative or neutral opinion.

Word lists:

```
POSITIVE = {    "good", "clean", "fast", "helpful",
"improved"}NEGATIVE = {    "bad", "dirty", "slow", "rude", "smoke",
"broken"}NEGATORS = {    "not", "never", "no"}
```

Python program:

```
import rePOSITIVE = {    "good", "clean", "fast", "helpful",
"improved"}NEGATIVE = {    "bad", "dirty", "slow", "rude", "smoke",
"broken"}NEGATORS = {"not", "never", "no"}def score(text):    words =
re.findall(r"[a-zA-Z]+", text.lower())    total = 0    negate = False
for word in words:        if word in NEGATORS:            negate = not
negate            continue        value = 1 if word in POSITIVE else
(            -1 if word in NEGATIVE else 0        )        if negate:
value = -value        total += value        negate = False
```

Expected output:

```
positive | Roads are clean and the bus was fast.
negative | Smoke from the factory, air is bad.
negative | The staff were not helpful.
positive | Not bad, service improved.
```

2. Extractive summariser

An extractive summariser selects important original sentences rather than generating new sentences.

Python program:

```
import refrom collections import CounterSTOPWORDS = {    "the", "is",
"a", "an", "and", "of",    "to", "in", "was", "are", "from",    "it",
"for", "on", "with", "this"}def summarise(text, number=2):
sentences = re.split(        r'(?<=[.!?])\s+', text.strip()    )
words = re.findall(r"[a-zA-Z]+", text.lower())    important =
[        word for word in words        if word not in STOPWORDS    ]
frequency = Counter(important)    scored = []    for index, sentence
in enumerate(sentences):        tokens = re.findall(            r"[a-
zA-Z]+", sentence.lower()        )        value =
sum(            frequency[word]            for word in tokens
if word not in STOPWORDS
```

Explanation:

- Splits the complaint into sentences.
- Counts frequently occurring important words.
- Calculates a score for each sentence.
- Selects the highest-scoring sentences.
- Displays the selected sentences in their original order.

A5.2 – Basic Chatbot (2 Marks)

Question: Design a rule-based chatbot that answers questions such as "What is the PM2.5 in Adyar today?" by matching keywords and querying the database.

Python program

```
import sqlite3def chatbot(question):    question = question.lower()
if "pm2.5" in question or "pm25" in question:        words =
question.replace("?", "").split()        station = None        for
word in words:            if word in ("adyar", "guindy"):
station = word.title()                break        if station is None:
return "Please mention the station name."        try:            with
sqlite3.connect("airquality.db") as con:                result =
con.execute("""                    SELECT r.pm25
FROM readings r                    JOIN stations s
ON r.station_id = s.station_id                    WHERE s.station_name
= ?                    ORDER BY r.date DESC                    LIMIT 1
""", (station,)).fetchone()            if result is None:
return "No reading available."
```

Expected output (if the latest Adyar reading is 48):

```
PM2.5 in Adyar is 48.0.
```

Limitations of a rule-based chatbot

1. It understands only predefined keywords.
2. It cannot understand complex questions.
3. It may fail with spelling mistakes.
4. It cannot understand context.
5. New questions require additional rules.

Alternative Concept and Complexity Analysis

If a machine-learning model such as Naive Bayes or a pretrained language model were used:

Rule-based chatbot	Machine-learning chatbot
Uses predefined rules	Learns patterns from data
Simple implementation	More complex implementation
Low memory requirements	Higher memory requirements
Limited question handling	Can handle more varied language
Predictable responses	Accuracy depends on model and data

Conclusion: A rule-based chatbot is suitable for simple queries. A machine-learning model may provide better language understanding but requires more resources and evaluation.

PART B – IMPLEMENTATION AND EXPERIMENTAL INVESTIGATION (25 MARKS)

Part B requires actual execution and measurement.

The workflow is:

ACQUIRE → CLEAN → STORE → QUERY → VISUALISE → PROFILE → CHALLENGE → INTERPRET

TASK B1 – IMPLEMENT AND VALIDATE THE CLEANING PIPELINE (7 MARKS)

B1.1 – Validate the Implementation (2 Marks)

Question: Implement the cleaning rules and test them on a small file containing every problem listed in A1. Show the data before and after cleaning.

Complete Python program

```
import csvimport ioRAW = """station,date,pm25Adyar,2026-10-
01,48Adyar,2026-10-02,NAAdyar,2026-10-03,Guindy,2026-10-
01,95Guindy,2026-10-02,-5Guindy,2026-10-03,121Guindy,2026-10-
04,1005"""def load_clean(text):    rows = []    previous = {}
reader = csv.DictReader(io.StringIO(text))    for row in reader:
station = row["station"].strip().title()        date =
row["date"].strip()        value = row["pm25"].strip()        # Skip
missing and invalid values        if value in ("", "NA"):
continue        try:            pm25 = float(value)        except
ValueError:            continue        # Skip negative values and
values above 999
```

Expected output

```
CLEAN DATA:

[
 ('Adyar', '2026-10-01', 48.0),
 ('Guindy', '2026-10-01', 95.0),
 ('Guindy', '2026-10-03', 121.0)
]

NORMALISED DATA:

[0.0, 0.644, 1.0]
```

Explanation:

- Missing values are skipped.
- NA values are skipped.
- Negative PM2.5 values are rejected.
- Values above 999 are rejected.
- Station names are standardised.
- Duplicate station-date records are removed.
- Valid readings are normalised using min-max scaling.

Note: This implementation uses a simple duplicate search for demonstration. The improved set-based approach in the next task is more efficient.

B1.2 – Compare with Your Design (1 Mark)

Answer:

The implementation follows the Part A design by reading CSV data, removing invalid readings and normalising PM2.5 values.

The implementation uses the CSV module instead of manually splitting strings. It also uses a set-based duplicate check in the improved version, reducing unnecessary searches.

B1.3 – Store and Query (2 Marks)

Question: Load the cleaned data into SQLite using `executemany()` and answer the three questions from A1.3.

Python program

```
import sqlite3clean = [    ("Adyar", "2026-10-01", 48.0),
("Guindy", "2026-10-01", 95.0),    ("Guindy", "2026-10-03", 121.0)]con
= sqlite3.connect("airquality.db")cur =
con.cursor()cur.execute("""CREATE TABLE IF NOT EXISTS stations
(    station_id INTEGER PRIMARY KEY,    station_name TEXT UNIQUE NOT
NULL)""")cur.execute("""CREATE TABLE IF NOT EXISTS readings
(    reading_id INTEGER PRIMARY KEY,    station_id INTEGER,    date
TEXT,    pm25 REAL,    UNIQUE(station_id, date),    FOREIGN
KEY(station_id)        REFERENCES stations(station_id))""")for station
in set(row[0] for row in clean):    cur.execute(        "INSERT OR
IGNORE INTO stations(station_name) VALUES (?)",        (station,)
```

Expected output

```
Adyar average:
48.0

Station averages:
[('Adyar', 48.0), ('Guindy', 108.0)]

Maximum reading:
('Guindy', 121.0)
```

B1.4 – Visualise (2 Marks)

Question: Produce charts with labelled axes and titles, and interpret each chart in two or three sentences.

Python chart program

```
import matplotlib.pyplot as pltfrom collections import
defaultdictclean = [    ("Adyar", "2026-10-01", 48.0),    ("Guindy",
"2026-10-01", 95.0),    ("Guindy", "2026-10-03", 121.0)]# Chart 1:
PM2.5 trendplt.figure()for station in set(row[0] for row in clean):
dates = [r[1] for r in clean if r[0] == station]    values = [r[2] for
r in clean if r[0] == station]    plt.plot(dates, values, marker="o",
label=station)plt.title("PM2.5 Trend by
Station")plt.xlabel("Date")plt.ylabel("PM2.5")plt.legend()plt.xticks(r
otation=45)plt.tight_layout()plt.show()# Chart 2: Station
comparisonaverages = defaultdict(list)for station, date, pm25 in
clean:    averages[station].append(pm25)stations = list(averages)means
= [
```

Chart interpretation

Chart 1 – PM2.5 Trend

The chart displays PM2.5 readings across the available dates. Guindy has higher readings than Adyar in the sample data.

Chart 2 – Station Comparison

The average PM2.5 in Guindy is 108, whereas Adyar has an average of 48. Guindy has the higher average in this sample.

Chart 3 – Distribution

The box plot shows the distribution of valid PM2.5 values. It helps identify the median and spread of the readings.

Note: The sample contains very few observations. These interpretations are demonstrations, not conclusions about actual city-wide air quality.

TASK B2 – ENGINEER ADVERSARIAL DATA (5 MARKS)

B2.1 – Test Different File Problems (2 Marks)

Question: Give the program files with missing columns, wrong delimiters, different encoding and empty files. Show how it responds.

Python program

```
import csvimport iodef test_file(text):    try:        if not
text.strip():            raise ValueError("Empty file")        reader
= csv.DictReader(io.StringIO(text))        required = {"station",
"date", "pm25"}        if not reader.fieldnames or not
required.issubset(            set(reader.fieldnames)        ):
raise ValueError("Missing or incorrect columns")        for row in
reader:            print(row)        print("File processed
successfully")    except (ValueError, csv.Error) as error:
print("Error:", error)# Test 1: Missing
columntest_file("station,date\nAdyar,2026-10-01")# Test 2: Wrong
delimitertest_file("station;date;pm25\nAdyar;2026-10-01;48")# Test 3:
Empty filetest_file("")
```

Expected output:

```
Error: Missing or incorrect columns
Error: Missing or incorrect columns
Error: Empty file

{'station': 'Adyar',
 'date': '2026-10-01',
 'pm25': '48'}

File processed successfully
```

Encoding test:

A file saved in an unsupported encoding may raise `UnicodeDecodeError` when read using an incompatible encoding. The program should catch this exception and record the filename and error.

B2.2 – Test SQL Injection (1 Mark)

Question: Search for a station name containing a quote character. First build the SQL string using `+`, then use a `?` placeholder.

Unsafe SQL query

```
station = "Adyar' OR '1'='1"query = (    "SELECT * FROM stations WHERE
station_name = '"    + station + "'")print(query)
```

Problem:

The input changes the SQL condition. The expression `'1'='1'` is always true, so the query may return unintended rows.

Safe SQL query

```
import sqlite3con = sqlite3.connect(":memory:")con.execute("""CREATE
TABLE stations (    station_id INTEGER PRIMARY KEY,    station_name
TEXT)""")con.executemany(    "INSERT INTO stations(station_name)
VALUES (?)",    [("Adyar",), ("Guindy",)])station = "Adyar' OR
'1'='1"result = con.execute(    "SELECT * FROM stations WHERE
station_name = ?",    (station,)).fetchone()print(result)con.close()
```

Expected output:

```
None
```

Explanation:

The parameterised query treats the input as ordinary text instead of executable SQL. Therefore, the injection attempt cannot change the query structure.

B2.3 – Test Sentiment with Difficult Complaints (1 Mark)

Test the sentiment scorer using:

1. Sarcasm.
2. Double negatives.
3. Mixed opinions.

Test program

```
complaints = [    "Great! Another day of dirty air.",    "Not bad, but the road is dirty.",    "The service is good but the air is bad.", "The staff are never helpful."]for complaint in complaints: print(complaint, "=>", label(complaint))
```

Expected observations:

Complaint	Expected basic interpretation
Great! Another day of dirty air.	Negative
Not bad, but the road is dirty.	Mixed
Service is good but air is bad.	Mixed
Staff are never helpful.	Negative

The simple word-list scorer may incorrectly label sarcasm or mixed opinions because it does not understand context.

B2.4 – Explain What Adversarial Input Reveals (1 Mark)

Adversarial input	What it reveals
Missing column	Whether schema validation works
Wrong delimiter	Whether CSV format is checked
Incorrect encoding	Whether decoding errors are handled
Empty file	Whether empty input is detected
SQL injection	Whether parameterised queries are used
Sarcasm	Limitations of word-list sentiment analysis
Mixed opinions	Limitations of a single overall sentiment score

Conclusion: Adversarial testing improves the reliability, security and accuracy of the application.

Alternative Concept and Complexity Analysis

Using parameterised SQL queries is safer than joining strings.

- String construction can produce SQL injection vulnerabilities.
- Parameterised queries separate SQL instructions from user input.

- `executemany()` is more efficient than executing a separate insert statement for every record.

TASK B3 – LOOPS VS LIBRARIES VS SQL (8 MARKS)

B3.1 – Correctness on Small Data (2 Marks)

Question: Compute the daily mean PM2.5 per station using three methods:

1. Plain Python loops with CSV.
2. Pandas `groupby`.
3. SQL `GROUP BY`.

All three methods should produce the same result.

Sample input

```
station,date,pm25
Adyar,2026-10-01,48
Adyar,2026-10-01,60
Guindy,2026-10-01,95
Guindy,2026-10-01,121
```

Method 1 – Python loops

```
import csvfrom collections import defaultdictgroups =
defaultdict(list)with open("readings.csv", "r") as file:    reader =
csv.DictReader(file)    for row in reader:        key =
(row["station"], row["date"])
groups[key].append(float(row["pm25"]))for key, values in
groups.items():    print(key, sum(values) / len(values))
```

Output:

```
('Adyar', '2026-10-01') 54.0
('Guindy', '2026-10-01') 108.0
```

Method 2 – Pandas groupby

```
import pandas as pddf = pd.read_csv("readings.csv")result =
df.groupby(    ["station", "date"])["pm25"].mean()print(result)
```

Output:

```
station  date
Adyar    2026-10-01     54.0
Guindy   2026-10-01    108.0
```

Method 3 – SQL GROUP BY

```
import sqlite3con = sqlite3.connect(":memory:")con.execute("""CREATE
TABLE readings (    station TEXT,    date TEXT,    pm25 REAL)""")data
= [    ("Adyar", "2026-10-01", 48),    ("Adyar", "2026-10-01", 60),
("Guindy", "2026-10-01", 95),    ("Guindy", "2026-10-01",
121)]con.executemany(    "INSERT INTO readings VALUES (?, ?, ?)",
data)result = con.execute("""SELECT station, date, AVG(pm25)FROM
readingsGROUP BY station, date""").fetchall()print(result)con.close()
```

Output:

```
[
 ('Adyar', '2026-10-01', 54.0),
 ('Guindy', '2026-10-01', 108.0)
]
```

Conclusion: All three methods produce the same daily mean values for the sample dataset.

B3.2 – Increase the Data Size (2 Marks)

Question: Generate data for 1, 12 and 120 stations over a year and measure the time of each method.

One station has approximately 8,760 hourly readings per year.

Stations	Approximate readings
1	8,760
12	105,120
120	1,051,200

Python benchmarking program

```
import timeimport randomimport sqlite3from collections import
defaultdictdef generate_data(stations):    data = []    for station in
range(stations):        for day in range(365):            for hour in
range(24):                value = round(random.uniform(10, 150), 2)
date = f"2026-{day + 1:03d}"
data.append((                    f"Station{station}",
date,                    value                ))    return datadef
python_method(data):    groups = defaultdict(list)    for station,
date, pm25 in data:        groups[(station, date)].append(pm25)
return {        key: sum(values) / len(values)        for key, values
in groups.items()    }
```

Expected observation:

The execution time increases as the number of stations and readings increases. Actual timings depend on the computer, Python version, SQLite version and available memory.

Performance comparison

Method	Advantage	Limitation
Python loops	Easy to understand	More manual coding
Pandas	Fast vectorised analysis	Higher memory usage
SQL	Efficient database aggregation	Requires database setup

B3.3 – Profile (2 Marks)

Question: Profile the plain-Python version with cProfile and identify the function that takes the most time.

Python program

```
import cProfileimport pstatsdef main():    data = generate_data(12)
result = python_method(data)    print("Total groups:",
len(result))profiler =
cProfile.Profile()profiler.enable()main()profiler.disable()stats =
pstats.Stats(profiler)stats.sort_stats("cumtime")stats.print_stats(10)
```

Explanation:

`cProfile` records how much time each function takes.

It helps identify slow operations such as:

- Reading large datasets.
- Creating dictionaries.
- Grouping records.
- Performing calculations.

The function with the highest cumulative time is a candidate for optimisation. The actual bottleneck must be identified from the measured profile rather than assumed.

B3.4 – Engineering Trade-Off (2 Marks)

Question: Explain the differences and state which method you would use for a quick one-time analysis and for a daily automated report.

Answer

Method	Suitable use
Python loops	Small one-time tasks and learning
Pandas	Quick analysis and data exploration
SQLite SQL	Daily automated reports and repeated queries

Quick one-time analysis:

Pandas is a convenient choice because it supports filtering, grouping and statistical calculations with fewer lines of code.

Daily automated report:

SQLite is a good choice because the data can be stored permanently and queried repeatedly.

Alternative Concept and Complexity Analysis

If each station's daily mean were calculated using a running sum and count instead of storing all readings:

```
summary = {}for station, date, pm25 in data:    key = (station, date)
if key not in summary:        summary[key] = [0, 0]    summary[key][0]
```

```
+= pm25    summary[key][1] += 1for key, (total, count) in
summary.items():    print(key, total / count)
```

Complexity comparison:

Method	Time	Space
Store all readings, then calculate	O(n)	O(n)
Running sum and count	O(n)	O(g)

Where:

- n = total number of readings.
- g = number of station-date groups.

Conclusion: Running sums and counts reduce memory usage because the program stores only the summary information rather than every reading.

TASK B4 – FINAL ENGINEERING DECISION (5 MARKS)

B4.1 – Implement a Practical Strategy (2 Marks)

Question: Implement the sentiment scorer and summariser on at least 50 complaints and compare their output with your own reading of the complaints.

Python program

```
import refrom collections import CounterPOSITIVE = {    "good",
"clean", "fast", "helpful", "improved"}NEGATIVE = {    "bad", "dirty",
"slow", "rude", "smoke", "broken"}NEGATORS = {"not", "never",
"no"}STOPWORDS = {    "the", "is", "a", "an", "and", "of",    "to",
"in", "was", "are", "from"}def score(text):    words =
re.findall(r"[a-zA-Z]+", text.lower())    total = 0    negate = False
for word in words:        if word in NEGATORS:            negate =
True            continue        value = 1 if word in POSITIVE else
(            -1 if word in NEGATIVE else 0        )        if negate:
```

Expected sentiment counts

For this test dataset:

Sentiment	Count
Positive	15
Negative	35
Neutral	0
Total	50

The counts are based on the simple word-list rules used in the program.

Comparison with human interpretation

Complaint	Program result	Human interpretation
Roads are clean and service is good	Positive	Positive
Smoke makes the air bad	Negative	Negative
Garbage collection is slow	Negative	Negative
Staff are helpful	Positive	Positive
Drainage system is broken	Negative	Negative
Air quality has improved	Positive	Positive

Conclusion:

The sentiment scorer can identify simple positive and negative complaints. However, sarcasm, mixed opinions and complex sentences may produce incorrect results. Human verification is necessary before using the results for important decisions.

B4.2 – Check the Database (1 Mark)

Question: Add an index on `(station, day)` and measure the effect on the slowest query.

SQL command

```
CREATE INDEX idx_station_date
ON readings(station_id, date);
```

PYTHON PROGRAM
C0_5

Python benchmarking program

```
import sqlite3import timecon = sqlite3.connect("airquality.db")query =
"""SELECT AVG(pm25)FROM readingsWHERE station_id = 1AND date BETWEEN
'2026-10-01' AND '2026-10-31'"""# Measure before indexstart =
time.perf_counter()con.execute(query).fetchone()before =
time.perf_counter() - start# Create indexcon.execute("""CREATE INDEX
IF NOT EXISTS idx_station_dateON readings(station_id,
date)""")con.commit()# Measure after indexstart =
time.perf_counter()con.execute(query).fetchone()after =
time.perf_counter() - startprint("Before index:", before)print("After
index:", after)
```

Expected observation:

An index can reduce the amount of data SQLite needs to search for matching station and date values.

Actual timing results depend on dataset size, storage hardware and caching. For a very small dataset, the index may provide little improvement.

B4.3 – Production Decision (2 Marks)

Question: Decide how the data should be stored, cleaned and analysed at the new scale.

Answer

The board will add 100 stations reporting every 15 minutes and receive 50,000 complaints per month.

1. Data storage

Use SQLite for structured air-quality readings and citizen complaints at the initial scale.

Store:

- Station information.
- Date and time of readings.
- PM2.5 measurements.
- Citizen complaints.

- Sentiment labels.
- Error logs.

Keep original CSV files as archived source data.

2. Data cleaning

- Remove duplicate readings.
- Reject negative and invalid PM2.5 values.
- Handle missing readings according to defined rules.
- Standardise station names and dates.
- Validate complaint text.
- Record rejected records in an error log.

3. Data analysis

- Use SQL `GROUP BY` for station summaries.
- Use Python for sentiment analysis.
- Use pandas for data exploration.
- Use Matplotlib for charts.
- Use indexes for frequently executed database queries.

5. Final engineering decision

Requirement	Recommended solution
Data acquisition	CSV and Python
Data cleaning	Python and pandas
Data storage	SQLite
Repeated queries	SQL
Sentiment analysis	Python word-list model initially
Summarisation	Extractive summariser
Visualisation	Matplotlib
Performance testing	cProfile and time.perf_counter
Error handling	Logging and specific exceptions

Conclusion:

I recommend storing cleaned air-quality readings and citizen complaints in SQLite, using Python for cleaning and sentiment analysis, and using SQL for repeated reports. Indexes should be

created on frequently queried columns. The application should be tested regularly as the number of stations and complaints increases.